

Tarea 3

Typechecking y Pattern Matching

Para la resolución de la tarea recuerde que

- Toda función debe estar acompañada de su firma, una breve descripción coloquial (en `T2.rkt`) y un conjunto significativo de tests (en `test.rkt`).
- Todo datatype definido por el usuario (via `deftype`) debe estar acompañado de una breve descripción coloquial y de la gramática BNF que lo genera.

Si la función o el datatype no cumple con estas reglas, será ignorado.

Ejercicio 1

30 Pt

- (a) [2 Pt] En el archivo `T3.rkt` se encuentra la definición del tipo de datos *Type*, que representa los tipos válidos para un lenguaje con valores numéricos, booleanos y funciones de 1° clase (por simplicidad de un solo argumento). Siguiendo los ejemplos, defina la función recursiva

```
;; parse-type :: <s-type> -> Type

>>> (parse-type 'Number)
(numT)
>>> (parse-type 'Boolean)
(boolT)
>>> (parse-type '(-> Number Boolean))
(arrowT (numT) (boolT))
>>> (parse-type 'String)
parse-type: invalid type String
```

- (b) [4 Pt] En el mismo archivo, se encuentra la definición base de un lenguaje con valores numéricos y booleanos. Extienda la gramática BNF del lenguaje y el tipo de datos *Expr* para que este soporte la definición y aplicación de funciones de 1° clase. Considere que como se quiere implementar *typechecking*, al definir funciones se debe indicar explícitamente el tipo del argumento. Puede guiarse con los siguientes ejemplos para la sintaxis concreta y abstracta (aún no necesita definir la función `parse`):

```
>>> (parse '(fun x Number (+ x 1)))
(fun 'x (numT) (add (id 'x) (num 1)))
>>> (parse '((fun x Number (* x 2)) 10))
(app (fun 'x (numT) (mul (id 'x) (num 2))) (num 10))
```

(c) [7 Pt] Escriba la función

```
;; parse :: <s-expr> -> Expr
```

que transforma un programa en sintaxis concreta a uno en sintaxis abstracta.

(d) [7 Pt] Implemente la función

```
;; check-type :: Expr TypeEnv -> Type
```

que realiza la comprobación de tipos (*typechecking*) sobre un programa en sintaxis abstracta. Para ello, considere las siguientes reglas:

- Las **constantes numéricas** son de tipo numérico (`numT`).
- Las **constantes** booleanas (`tt` y `ff`) son de tipo *boolean* (`boolT`).
- En el caso de la **suma** y la **multiplicación**, estas tienen tipo numérico (`numT`), si y solo si, ambos operandos tienen tipo numérico (`numT`). Por otro lado, la operación “**menor que**” es de tipo *boolean* (`boolT`) si ambos operandos son de tipo numérico.
- Un identificador obtiene su tipo a partir de la extracción de un ambiente de tipos (`TypeEnv`).
- En el caso de los **condicionales** (`if`), la condición debe tener tipo `boolT`. Por otro lado, ambas ramas de ejecución deben tener el mismo tipo entre sí, siendo este el tipo que recibe el condicional completo.
- Las funciones tienen su propio tipo (`arrowT`) que indica el tipo del dominio y el tipo del codominio. En el caso del primero, las funciones son declaradas con el tipo del argumento, por lo que se conoce de antemano. Por otro lado, para el codominio se debe inferir el tipo extendiendo el ambiente de tipos con el argumento de la función.
- La aplicación de una función tiene el tipo del codominio si el argumento que se está entregando es del tipo indicado en la declaración de la función.

Cuando las reglas anteriores no se cumplen, la función debe arrojar errores, para ello, siga estos ejemplos:

```
>>> (check-type (add (tt) (num 50)) empty-tenv)
check-type: Addition expects 2 numbers
>>> (check-type (mul (num 10) (ff)) empty-tenv)
check-type: Multiplication expects 2 numbers
>>> (check-type (lt (num 25) (ff)) empty-tenv)
check-type: Less-than expects 2 numbers
>>> (check-type (ite (num 100) (tt) (ff)) empty-tenv)
check-type: If condition must be a boolean
>>> (check-type (ite (tt) (num 500) (ff)) empty-tenv)
```

```
check-type: If branches type mismatch
>>> (check-type (app (num 100) (num 300)) empty-tenv)
check-type: Function application to a non-function
>>> (check-type (app (fun 'x (numT) (mul (id x) (num 2))) (ff)) empty-tenv)
check-type: Function argument type mismatch
```

- (e) [7 Pt] Escriba la función

```
;; interp :: Expr Env -> Value
```

que evalúa un Expr hacia un valor final del lenguaje. Asuma que el programa (Expr) está bien tipado.

- (f) [3 Pt] Implemente la función

```
;; run :: <s-expr> -> Value
```

que reduce programas en sintaxis concreta a valores del lenguaje, si y solo si, el programa está bien tipado.

Hint: Le puede ser de utilidad la función `begin` de Racket.

Ejercicio 2

30 Pt

- (a) [6 Pt] En el mismo archivo (T3.rkt) se define el tipo de datos Prog como un lenguaje básico con números, adición, un constructor de pares (conz) y el constructor nil. Implemente la función

```
;; parser :: <s-prog> -> Prog
```

e incluya azúcar sintáctico para definir listas a partir de los constructores conz y nil. Puede seguir los ejemplos:

```
>>> (parser '(nil))
(nil)
>>> (parser '(cons 1 2))
(conz (numbr 1) (numbr 2))
>>> (parser '(list 1 2 3))
(conz (numbr 1) (conz (numbr 2) (conz (numbr 3) (nil))))
```

- (b) [4 Pt] Como se va a implementar *pattern matching*, se define el tipo de datos Pattern. Escriba la función

```
;; parse-pattern :: <s-prog> -> Pattern
```

siguiendo los ejemplos:

```
>>> (parse-pattern '3)
(numP 3)
>>> (parse-pattern 'x)
(varP 'x)
>>> (parse-pattern '(nil))
(nilP)
>>> (parse-pattern '(cons 1 2))
```

```
(conzP (numP 1) (numP 2))
>>> (parse-pattern '(list 1 2 3))
(conzP (numP 1) (conzP (numP 2) (conzP (numP 3) (nilP))))
```

- (c) [6 Pt] El desafío de implementar un lenguaje con *pattern matching* es determinar si un valor calza con un patrón dado, encontrando además la asociación entre variable y valor. Por ejemplo, el valor `(cons (1 (cons 2 3)))` calza con el patrón `(cons x y)`, en donde `x` se asocia con `1` e `y` con `(cons 2 3)`. Por otro lado, el valor `(cons 2 3)` no calza con el patrón `(cons x (nil))`, pues `2` no corresponde con el patrón `(nil)`.

En T3.rkt se define el tipo de datos `Result` que permite retornar un resultado exitoso o un mensaje de error sin la necesidad de arrojar una excepción. De esta manera, se puede hacer un manejo explícito de los errores. Por ejemplo, se puede definir una “división segura”:

```
;; safe-div :: Number Number -> (Result String Number)
(define (safe-div x y)
  (if (zero? y) (failure "DivisionByZero") (success (/ x y))))
```

Bajo esta idea, implemente la función

```
;; generate-substs :: Pattern PValue -> (Result String (Listof (Symbol *
  PValue)))
```

que a partir de un patrón y un valor, retorna una lista de substituciones inducidas por el *pattern matching* o un mensaje de error si el *match* no es exitoso. Para ello, asuma que:

- `nil` es un valor (`nilV`)
- Un par construido con `conz` es un valor (`conzV`) solo si sus componentes son valores.
- En un patrón nunca se repite el nombre de una variable.

```
>>> (generate-substs (numP 3) (numerical 3))
(success '())
>>> (generate-substs (numP 3) (numerical 4))
(failure "MatchError: given number does not match pattern")
>>> (generate-substs (varP 'x) (numerical 3))
(success (list (cons 'x (numV 3))))
>>> (generate-substs (numP 3) (nilV))
(failure "MatchError: expected a number")
>>> (generate-substs (nilP) (numerical 45))
(failure "MatchError: expected nil")
>>> (generate-substs (conzP (varP 'x) (nilP)) (numerical 22))
(failure "MatchError: expected a cons constructor")
```

- (d) [6 Pt] Escriba la función

```
;; reduce :: Prog (Listof (Symbol * PValue)) -> PValue
```

recordando las características de los valores (PValue) mencionados en la sección anterior.

- (e) [6 Pt] Extienda la gramática BNF del lenguaje, el tipo Prog, la función parser y la función reduce para que soporten la expresión `match`. Siga los ejemplos:

```
>>> (parser '(match 2 [1 1] [x 3]))
(pmatch (numbr 2) (list (cons (numP 1) (numbr 1)) (cons (varP 'x) (numbr 3))))
>>> (parser '(match (list 1 2 3)))
SyntaxError: match expression must have at least one case
>>> (reduce (pmatch (numbr 2) (list (cons (numP 1) (numbr 1)) (cons (varP 'x) (
  numbr 3)))) '())
(numerical 3)
>>> (reduce (pmatch (nil) (list (cons (numP 1) (numbr 10)) (cons (numP 2) (
  numbr 20)))) '())
MatchError: expression does not match any pattern
```

- (f) [2 Pt] ¿Por qué es útil que `generate-substs` no arroje una excepción sino que retorne un mensaje de error?